

Apéndice técnico

Gold Dice RL — Trabajo Práctico 1 — Aprendizaje por Refuerzos

El informe tiene un tope de tres páginas. Acá está el material que no entra: el análisis del ambiente línea por línea, y la tabla completa de resultados con sus intervalos de confianza.

Nada de esto hace falta para leer el informe, y todo es reproducible con los scripts del paquete.

Gold Dice RL — las reglas que el enunciado no cuenta

Este documento sale de leer `env.py` y `config.py` línea por línea, no de leer el PDF. El enunciado describe el juego. El código es el juego. Cuando difieren, gana el código. Todo lo que está acá está verificado contra el código fuente y, donde correspondía, contra el solver exacto (`oracle_dp.py`). Nada es una intuición.

0. El resumen en una frase

El juego parece "tirar dados y decidir en qué gastar". En realidad es un problema de **control óptimo estocástico con horizonte finito, una sola acción por turno y un activo riesgoso**. Y —esto es lo importante— es **lo bastante chico como para resolverlo exacto**. Sabemos cuál es la respuesta perfecta: **642.45 puntos esperados**. Todo el trabajo práctico se mide contra ese número.

Para referencia inmediata:

Agente	Puntaje medio (20.000 partidas, banda desarrollo)	% del óptimo
Óptimo teórico (programación dinámica exacta)	642.45	100 %
SimpleExpectancy (baseline provisto por la cátedra)	343.4	53 %
RandomLegal (baseline provisto)	63.0	10 %

El baseline "bueno" de la cátedra deja **casi la mitad de los puntos sobre la mesa**. Eso no es un detalle: es el espacio entero donde vive este trabajo práctico.

1. El orden de operaciones dentro de un turno

Esto es lo primero que hay que fijar, porque de acá salen la mitad de los casos borde.

Leyendo `env.step()` → `_apply_action()` → `_apply_storm()` → `_advance_turn()` → `_roll_for_current_turn()`, la secuencia real es:

```
TURNO t
1. TIRADA      gold += roll_sum + stored_value ; stored_value = 0
2. OBSERVACIÓN el agente ve el estado (el oro de la tirada YA está sumado)
3. ACCIÓN      exactamente UNA acción
4. TORMENTA    con p = 0.15
5. t := t + 1 ; si t > 30 → fin (y NO se vuelve a tirar)
```

Escrito así, tres cosas saltan a la vista. Ninguna de las tres está en el PDF.

2. Caso borde #1 — La tormenta llega después de tu acción

`_apply_action` corre antes que `_apply_storm`. Siempre.

Consecuencia: los puntos son inmunes. La tormenta solo puede tocar el **oro que decidiste no puntuar**. Si en un turno puntuás todo tu oro, la tormenta de ese turno te pega a un cofre vacío y no te hace absolutamente nada.

La analogía: el oro es plata en efectivo arriba de la mesa; los puntos son plata en el banco. El ladrón pasa una vez por turno, después de que vos hiciste tu movida. Si depositaste, no encuentra nada.

Esto reordena por completo la lógica del escudo. El escudo **no** te protege del riesgo del juego: te protege del riesgo de una decisión que ya tomaste voluntariamente (guardar oro). Un agente que compra escudos "por las dudas" mientras puntúa todo cada turno está quemando 5 de oro por nada.

Corolario duro: SCORE no es solo la forma de ganar puntos. Es el instrumento de cobertura de riesgo del juego. Es la única acción que convierte un activo volátil en uno seguro, y lo hace a paridad 1:1 sin comisión.

3. Caso borde #2 — El turno 30 no tiene tirada después

En `_advance_turn`:

```
self.turn += 1
if self.turn > HORIZON:
    self.done = True
    return self.get_obs(), reward, True, info      # ← sale ANTES de _roll_for_current_turn()
self._roll_for_current_turn()
```

Consecuencia: todo el oro que sobreviva al turno 30 sin haber sido puntuado **se pierde**. No se convierte, no se arrastra, no vale nada.

Y por lo tanto, en el turno 30:

- `BUY_DICE` → tirás entre 18 y 90 de oro a la basura. El dado nunca se tira.

- UPGRADE → lo mismo.
- BUY_SHIELD → comprás protección para un cofre que ya no importa.
- STORE_BEST_DIE → pagás 4 por un valor que se cobraría en el turno 31, que no existe.
- PASS → perdés todo el oro.
- SCORE(gold) → **única jugada no dominada**.

El solver exacto lo confirma sin que se lo digamos: $V^*(t=30, \text{gold}=g, \dots) = g$ exactamente, para todo n, b, s . La estructura del último turno no se aprende: se deduce, y conviene codificarla.

Nota práctica: SimpleExpectancy acierta este caso (if turns_left == 0: return SCORE, gold), pero por la razón equivocada — lo trata como "el final del plan", no como "el oro se evapora". Un agente tabular que agrupa el turno 30 con el 29 en el mismo bucket de estado pierde ~40 puntos de una y no se entera nunca.

4. Caso borde #3 — STORE_BEST_DIE clona un dado, no lo guarda

Este es el más contraintuitivo, y el que más gente lee mal.

El nombre sugiere "aparto este dado para usarlo después". Lo que el código hace es otra cosa. En roll_for_current_turn, la suma completa de la tirada —incluido el dado más alto— **ya se sumó al oro**:

```
self.gold += self.roll_sum + self.stored_value
```

Después, STORE_BEST_DIE solo hace:

```
self.gold -= STORE_DIE_COST      # -4
self.stored_value = self.roll_max
```

Y roll_max vuelve a entrar al oro en la tirada siguiente. O sea: **cobraste ese dado dos veces**. No es un traslado, es una fotocopia.

Economía de la jugada: pagás 4, cobrás roll_max el turno siguiente. Como roll_max incluye el bonus (current_roll = raw + dice_bonus), con bonus = 3 y 4 dados el valor esperado del máximo ronda 8.2, o sea **+4.2 de oro neto por 4 de costo**. Con el motor desarrollado es una de las acciones más rentables del juego, y es la que casi nadie usa.

Pero consume tu única acción del turno. El costo de oportunidad no es "lo que hubieras puntuado" (ver §5: eso es falso), sino la tormenta esperada sobre el oro que dejás expuesto: $0.075 \times \text{oro}$. Con poco oro, o detrás de un escudo, STORE_BEST_DIE es casi gratis y rinde. Con 400 de oro sin escudo cuesta ~30 puntos y no vale la pena.

Por eso el juego óptimo la usa muchísimo (§8-bis): **es la acción dominante en los turnos 1-13, cuando el oro es chico, y otra vez en los turnos 24-29, cuando ya hay un escudo puesto**.

5. Caso borde #4 — Una sola acción por turno (y qué cuesta de verdad)

step() toma una acción. Una. No podés puntuar y comprar en el mismo turno.

La lectura tentadora es: "entonces gastar un turno cuesta lo que hubieras puntuado". **Es falsa**, y el solver lo dice sin ambigüedad. No puntuar no destruye el oro: sigue ahí el turno siguiente, y un solo SCORE convierte *todo* tu oro de una. Lo único que perdés por no puntuar es la exposición a **una** tormenta.

Medido con el oráculo — $V^*(\text{puntuar todo})$ contra $V^*(\text{pasar})$ en el mismo estado:

turno	oro	escudos	$V^*(\text{SCORE})$	$V^*(\text{PASS})$	costo del turno	
27	300	0	437.71	414.95	22.76	= 7.59 % del oro
27	300	1	444.15	444.11	0.04	el escudo lo anula
20	150	0	516.62	493.69	22.93	
10	60	0	900.55	900.55	0.00	
3	20	0	759.51	759.51	0.00	

$0.15 \times 300/2 = 22.5$. La medición da 22.76. **El costo de gastar un turno es exactamente la tormenta esperada sobre el oro que dejás expuesto: 7.5 % de ese oro, y nada más.**

Y la segunda fila es la que importa: **con un escudo el costo cae a 0.04**. Para eso sirve el escudo. No te protege del juego: te compra el derecho a usar tus turnos en algo que no sea cobrar.

Dónde sí aparece la escasez de acciones

En el valor marginal del oro — cuántos puntos vale una moneda:

turno	10	30	60	120	250	← oro
2	13.898	11.693	4.330	0.955	0.368	
6	10.323	10.294	4.082	0.947	0.368	
14	5.022	1.080	1.422	0.811	0.977	
22	1.648	0.989	1.001	1.000	1.000	
30	1.000	1.000	1.000	1.000	1.000	

- **Turno 2, 10 de oro: una moneda vale 13.9 puntos.** Desbloquea la primera mejora (cuesta 8), y esa mejora rinde durante 28 turnos.
- **Turno 2, 250 de oro: una moneda vale 0.37 puntos.** Vale *menos* que un punto.

Ese segundo número es la escasez de acciones. Con 250 de oro en el turno 2 tenés **más plata que turnos para gastarla**: solo podés hacer una compra por turno, y el sobrante no hace nada más que juntar riesgo de tormenta. El cuello de botella no es "el turno vale lo que puntuarías", es **"no podés convertir oro en maquinaria más rápido que una compra por turno"**.

6. Caso borde #5 — SCORE no es una acción, son 401 (y por qué eso igual no importa)

`get_action_mask()` marca SCORE como siempre válida, con este comentario en el código:

```
# SCORE is valid because score_amount=0 is always legal.
```

Dos hechos:

(a) PASS es redundante. SCORE(0) hace exactamente lo mismo.

(b) El espacio de acciones real es paramétrico. `get_valid_score_amounts()` devuelve literalmente `range(gold + 1)`: con 400 de oro hay 404 acciones legales.

La hipótesis obvia, y por qué es falsa

La conclusión tentadora es: "un agente tabular que discretiza SCORE en {TODO, MITAD} tira el problema a la basura". Lo medimos re-resolviendo el juego entero con el espacio de SCORE mutilado a propósito:

Espacio de SCORE permitido	Valor óptimo	Costo
k libre en [0, oro] (401 acciones)	642.45	—
Solo SCORE(oro) — una sola acción	642.45	0.00
Solo SCORE(oro) o SCORE(oro//2)	642.45	0.00
Solo SCORE(oro//2) (control negativo)	641.33	1.12

Reducir SCORE de 401 acciones a una sola no cuesta nada. El control negativo confirma que la medición no es un artefacto: cuando sacamos la opción de puntuar todo, el valor sí cae.

Por qué no importa

Porque el juego óptimo **puntúa una sola vez, en el turno 30, y puntúa todo** (ver §9). Los estados donde puntuar parcial ayuda existen —a turno 18 con 188 de oro, guardar 30 en vez de puntuar todo vale 15.36 puntos— pero son estados a los que el juego óptimo **nunca llega**. Solo se visitan jugando mal.

Es un caso borde real del ambiente y hay que conocerlo. Pero como decisión de modelado está **medida en cero**, y eso permite reducir el espacio de acciones a **6 acciones discretas sin perder un solo punto**. Ese es un resultado útil: no porque revele una trampa, sino porque descarta una.

Anotarlo importa por método: la intuición decía que ésta era la decisión de modelado cara. El solver dice que vale 0.00. Cuando se puede medir, no se opina.

7. Caso borde #6 — La tormenta usa división entera

```
self.gold = self.gold // 2
```

- Con oro impar te quedás con el redondeo hacia arriba en la práctica (39 → 19, perdés 20).
- Con oro = 1 → 0.
- **Con oro = 0 la tormenta es gratis.** No hay penalización fija, no hay pérdida de puntos, no pasa nada.

Combinado con §2: **si tu oro es 0, comprar un escudo es tirar 5 monedas**. El escudo solo vale por el oro que efectivamente vas a tener expuesto en el momento en que caiga la próxima tormenta.

Cuándo conviene un escudo (cuenta rápida). Cuesta 5. La tormenta pega con $p = 0.15$ y te saca $\text{gold}/2$. Como pérdida esperada de un solo turno, conviene si $0.15 \cdot \text{gold}/2 > 5$, o sea $\text{gold} > 67$. Pero el escudo **no vence**: sigue ahí hasta que una tormenta lo consuma. Sobre T turnos restantes la probabilidad de que se use es $1 - 0.85^T$ (99 % con 28 turnos por delante), y lo que bloquea es la mitad del oro *en el momento de la tormenta*, no el de hoy. El umbral real es bastante más bajo que 67 y depende del horizonte. Es exactamente el tipo de cuenta que una fórmula cerrada hace mal y una tabla de valores hace bien.

8. Caso borde #7 — Las semillas no son comparables entre agentes

Este es el caso borde metodológico, y es el que más caro sale si se ignora.

`env.rng` es **un solo generador** que sirve dos cosas distintas:

```
self.raw_roll = self.rng.choice(DICE_FACES, size=self.num_dice, ...) # consume ~num_dice draws
...
if self.rng.random() >= STORM_PROB: return # consume 1 draw
```

`size=self.num_dice` **cambia con la política**. Un agente que compró 5 dados consume 5 draws por turno; uno que se quedó con 1 consume 1. A partir del primer turno en que dos agentes difieren en cantidad de dados, **sus flujos de aleatoriedad se desincronizan por completo**.

Consecuencia: correr dos agentes con `seed=0` **no** les da las mismas tiradas ni las mismas tormentas. La técnica estándar de *common random numbers* —comparar políticas sobre el mismo azar para cancelar varianza— **no está disponible acá**. No hay forma de conseguirla sin tocar `env.py`, que el enunciado prohíbe.

Qué implica en números. Con `n_episodes = 1000` y una desviación estándar típica de ~100 puntos, el error estándar de la media es $100/\sqrt{1000} \approx 3.2$ puntos. El intervalo de confianza al 95 % es de **±6.3 puntos**. Entonces:

- Una diferencia de 4 puntos entre dos agentes en el leaderboard **no es una diferencia**.
- Con desvíos altos (una política agresiva puede tener $\sigma \approx 150$) el intervalo se abre a ± 9 .
- El torneo final usa **semillas privadas**: cualquier ajuste fino contra $\text{seed}=0$ es sobreajuste a ruido.

Por eso todo resultado que reportemos va con $n = 20.000$ episodios e intervalo de confianza, no con $n = 1000$ y un número pelado. No es prolijidad: es la diferencia entre medir y adivinar.

8-bis. Qué hace realmente el juego óptimo (400 partidas del oráculo)

Todo lo anterior son piezas sueltas. Así se arman:

turno	oro	dados	bonus	escudos	acción dominante	
1-13	4 -> 30	1 -> 3.1	0 -> 2.3	0.00	STORE_BEST_DIE	(36-51 %)
14-22	34 -> 81	3.5 -> 7.2	2.6 -> 5.4	0.00	BUY_DICE	(36-68 %)
23	101	7.5	6.1	0.26	BUY_SHIELD	(61 %)
24-29	155 -> 459	7.5	6.1	-0.80	STORE_BEST_DIE	(75-86 %)
30	463	7.5	6.1	0.58	SCORE	(100 %)

Leído como estrategia:

1. **Turnos 1-13 - construir barato.** Casi no hay oro. STORE_BEST_DIE cuesta 4 y devuelve roll_max ; con poco oro expuesto la tormenta no duele. Se acumulan mejoras (baratas al principio: 8, 16, 24) más que dados. 2. **Turnos 14-22 - construir caro.** Ya entra oro suficiente para dados (42, 50, 58...). El motor llega a ~ 7.5 dados y bonus ~ 6 , o sea ~ 72 de oro por turno. 3. **Turno 23 - levantar el muro.** Un escudo. Uno solo. 4. **Turnos 24-29 - acumular detrás del muro.** El oro sube de 155 a 459 **sin puntuar**. Con escudo, no puntuar cuesta 0.04 puntos por turno (\$5), así que la acción libre se usa en STORE_BEST_DIE para seguir sumando. 5. **Turno 30 - cobrar.** Un único SCORE de ~ 463 de oro.

El juego óptimo casi no puntúa hasta el último turno. De ahí sale, retroactivamente, por qué la granularidad de SCORE no importa (\$6): se usa una vez, y se usa entera.

Por qué un solo escudo y no cinco: un escudo bloquea una tormenta. Entre el turno 23 y el 30 hay 7 tormentas potenciales a $p=0.15$, o sea ~ 1.05 esperadas. Comprar exactamente uno -y recomprarlo cuando se consume, que es por qué el promedio de escudos baja de 0.86 a 0.58- es la cobertura justa. Ni de más ni de menos.

El costo de no poder ver ese endgame

Esta trayectoria dice exactamente qué tiene que representar el estado:

Variable	Rango que visita el juego óptimo
num_dice	hasta 9 (mediana 8 al final)
dice_bonus	hasta 8 (mediana 6)
gold	hasta 797
shields	0 ó 1, casi nunca más

Una discretización que capee num_dice en 5, dice_bonus en 4 y el oro en 240 -una elección que parece razonable antes de mirar la trayectoria- colapsa todo el endgame en una sola celda de la tabla:

- **38.0 %** de las decisiones óptimas ocurren con $\text{num_dice} > 5$
- **35.7 %** ocurren con $\text{dice_bonus} > 4$
- el oro llega a 797, más de 3x el tope

Un agente con esos topes **no puede distinguir el turno 24 con 160 de oro del turno 29 con 500**, y ahí se decide el 70 % del puntaje. No aprende mal: no puede ver. Ninguna cantidad de episodios lo arregla.

9. La consecuencia constructiva: el juego se puede resolver exacto

Los casos borde anteriores no son curiosidades. Juntos dicen que este MDP es **chico**.

Observación clave — los puntos no son parte del estado. points no aparece en ninguna transición: no afecta la tirada, ni los costos, ni la tormenta. Solo se acumula. Entonces el retorno es separable y la función de valor **no depende de los puntos**:

$V_t(\text{gold}, \text{num_dice}, \text{dice_bonus}, \text{shields}) \leftarrow$ cuatro variables, no nueve

Esto saca points (no acotado) del estado de un plumazo. Y stored_value se puede plegar en la transición (entra como un corrimiento aditivo sobre el oro del turno siguiente), y roll_sum ya está adentro de gold en el momento de decidir. De nueve variables observadas quedan **cuatro que importan, más el roll_max del turno, que solo afecta a una acción**.

El truco que hace tratable a SCORE. Con $f(m)$ = valor de terminar el turno con m de oro:

$$\max_{\{0 \leq k \leq g\}} [k + f(g - k)] = g + \max_{\{m \leq g\}} [f(m) - m]$$

El término de la derecha es un **máximo prefijo**: se calcula una vez por estado, en $O(\text{oro})$, y sirve para todos los valores de g a la vez. La acción "continua" deja de ser un problema.

Resultado. Inducción hacia atrás sobre $t = 30 \dots 1$, con la distribución conjunta exacta de (suma, máximo) de n dados (calculada por convolución, no por muestreo):

$$V^*(t=1, \text{gold}=0, \text{dados}=1, \text{bonus}=0, \text{escudos}=0) = 643.46 \text{ puntos}$$

Verificado con topes $n \leq 12$, $\text{bonus} \leq 12$, $\text{escudos} \leq 8$, $\text{oro} \leq 1400$. Con topes más chicos (9/9/5/700) da 642.45 — una diferencia de 0.16 %, o sea los topes prácticamente no muerden y el número es sólido.

Para qué sirve tener el óptimo. No para entregarlo (el enunciado pide un agente *aprendido*). Sirve para tres cosas que sin él son imposibles:

1. **Un denominador honesto.** "Nuestro agente saca 610" no dice nada. "Nuestro agente captura el 95 % del óptimo teórico, contra el 54 % del baseline de la cátedra" dice todo. 2. **Diagnóstico por decisión.** Para cada jugada del agente aprendido podemos calcular el *arrepentimiento* exacto: $V^*(s) - [r + V^*(s')]$. Eso nos dice **en qué turnos y en qué estados** el agente se equivoca, en vez de solo cuánto. Es la diferencia entre "el modelo anda mal" y "el modelo sub-invierte en bonus entre los turnos 5 y 9". 3. **Separar habilidad de suerte** (en el juego web): la diferencia entre lo que valía tu partida y lo que sacaste es, exactamente, suerte.

10. Casos borde menores (verificados, sin impacto estratégico)

#	Observación en el código	Impacto
10.1	<code>stored_value</code> se sobrescribe (= <code>self.roll_max</code>), no se acumula. Como se consume en cada tirada, no se puede apilar de todas formas.	Ninguno.
10.2	El oro guardado se suma al inicio del turno siguiente, o sea queda expuesto a la tormenta de ese turno.	Chico; el solver lo tiene en cuenta.
10.3	<code>__init__</code> llama <code>default_rng(seed)</code> y después <code>reset(seed)</code> , que lo vuelve a llamar. Doble inicialización inofensiva.	Ninguno.
10.4	<code>roll_max</code> incluye el bonus; <code>raw_roll</code> no. <code>current_roll = raw + dice_bonus</code> .	Sube el valor de <code>STORE_BEST_DIE</code> con bonus alto.
10.5	Turno 1: 1 dado, bonus 0 $\rightarrow$ oro $\in \{1..6\}$. Todo cuesta ≥ 4 , así que la única jugada posible además de SCORE/PASS es <code>STORE_BEST_DIE</code> , y solo si salió 4, 5 o 6.	Micro-decisión de apertura real.
10.6	No hay tope de dados, mejoras ni escudos, pero los costos crecen linealmente (+8 cada uno) mientras el retorno por dado es constante ($3.5 + \text{bonus}$).	El tope es económico, no reglamentario. Aparece solo.
10.7	El horizonte es fijo y conocido (30 turnos), no hay terminación estocástica.	Justifica $\gamma = 1$. Ver §11.

11. Un caso borde que no está en el ambiente sino en cómo se lo modela: γ

El retorno real del problema es la suma **sin descontar** de los puntos: $G = \sum r_t$. El episodio termina siempre en 30 pasos, así que no hay ningún problema de convergencia que justifique descontar.

Usar $\gamma = 0.995$ —un valor que parece inocente— multiplica el pago del turno 30 por $0.995^{30} \approx 0.865$. Es decir: **le decís al agente que los puntos del final valen 13.5 % menos que los del principio**, en un juego cuyo único momento de cobro grande es el final. El descuento sesga sistemáticamente en contra de la estrategia correcta (invertir temprano, cobrar tarde) y es una de esas decisiones que se copian de un tutorial sin pensarlas.

En este problema $\gamma = 1$ es lo correcto y hay que decir por qué: horizonte finito, fijo, conocido, y episodio garantizado a terminar. El descuento acá no es un regularizador, es un sesgo.

12. Checklist: qué tiene que respetar cualquier agente que hagamos

De todo lo anterior, las condiciones que un agente debe cumplir para no estar roto de entrada:

- [] En $t = 30$ hace `SCORE(gold)`. Siempre. Sin excepción, sin aprenderlo.
- [] Representa `num_dice` hasta 9, `dice_bonus` hasta 8 y oro hasta ~800 **sin saturar**.
- [] Entrena con $\gamma = 1$.
- [] No compra escudos cuando el oro expuesto no lo justifica.
- [] Sabe que `STORE_BEST_DIE` duplica, y compite contra el costo de oportunidad del turno.
- [] Se evalúa con $n \geq 20.000$ episodios e intervalo de confianza, nunca con 1000 a secas.
- [] Se reporta como fracción del óptimo (642.45), no como número absoluto.

Resultados

Protocolo: $n = 20.000$ episodios por agente, intervalo de confianza al 95 %, banda de semillas desarrollo (base 1.000.000). Todas las decisiones de diseño se tomaron mirando esta banda. La banda pública (`seed 0`, la del enunciado) y la de control (2.000.000) se miran una sola vez, al cerrar. Reproducible con `python results.py desarrollo 20000`.

1. Tabla principal

Agente	Media	IC 95 %	% del óptimo	σ
Oráculo (DP exacta, <i>no compite</i>)	644.10	[642.23, 645.97]	100.3 %	134.9

Agente	Media	IC 95 %	% del óptimo	σ
GoldDiceAgent (Double TD sobre afterstates)	527.06	[525.16, 528.97]	82.0 %	137.2
Solo potencial Φ (cero aprendizaje)	490.41	[488.03, 492.79]	76.3 %	171.5
Q clásico — rangos amplios + SCORE_ALL	364.21	[363.69, 364.73]	56.7 %	37.8
Q clásico — rangos amplios, $\gamma=1$	354.83	[354.34, 355.32]	55.2 %	35.2
Q clásico — rangos amplios, $\gamma=0.995$	354.60	—	55.2 %	—
SimpleExpectancy (baseline de la cátedra)	343.40	[341.96, 344.85]	53.5 %	104.2
Q clásico — topes chicos, $\gamma=0.995$	314.58	[314.05, 315.11]	49.0 %	38.1
Q clásico — topes chicos, $\gamma=1$	292.63	[292.04, 293.22]	45.5 %	42.6
Double TD sin potencial (Φ apagado)	105.06	—	16.4 %	—
RandomLegal (baseline de la cátedra)	63.03	[62.50, 63.57]	9.8 %	38.8

El agente entregado saca **52 % más** que el mejor baseline provisto y **casi el doble** que un Q-Learning tabular armado con la receta estándar.

2. Ablaciones: qué cuesta cada decisión de modelado

Todas con el mismo presupuesto (400.000 episodios), cambiando **una** cosa por vez.

Cambio	De	A	Δ
Topes de estado: $\text{dados} \leq 5$, $\text{bonus} \leq 4$, $\text{oro} \leq 240 \rightarrow$ rangos que cubren el juego óptimo	292.63	354.83	+62.2
Espacio de SCORE: {TODO, MITAD} $\rightarrow$ {TODO}	354.83	364.21	+9.4
$\gamma = 0.995 \rightarrow 1$, con rangos amplios	354.60	354.83	+0.2 (nulo)
$\gamma = 0.995 \rightarrow 1$, con topes chicos	314.58	292.63	-21.9
Representación: tabla Q discretizada $\rightarrow$ afterstates + Φ + Double	364.21	527.06	+162.9
Potencial Φ : apagado $\rightarrow$ encendido	105.06	527.06	+422.0

2.1 Tres resultados que contradicen la intuición

(a) Mutilar SCORE casi no cuesta. La hipótesis de trabajo inicial era que colapsar una acción de 401 valores en dos era *el* error caro. Medido sobre el óptimo exacto (`oracle_dp.solve(score_mode=...)`), el costo es **0.00**: el juego óptimo puntúa una sola vez, en el turno 30, y puntúa todo. El control negativo confirma que la medición discrimina — prohibir "puntuar todo" y dejar solo "la mitad" sí cuesta 1.12 puntos.

En un agente subóptimo el efecto es algo mayor (+9.4) porque ese agente sí acumula oro en estados donde puntuar parcial ayuda. Pero no es la explicación de nada.

(b) Los topes de estado cuestan 62 puntos. El juego óptimo llega a 9 dados, bonus 8 y 797 de oro. Con $\text{dados} \leq 5$, $\text{bonus} \leq 4$, $\text{oro} \leq 240$, el **38 %** de las decisiones óptimas y todo el endgame caen en una única celda de la tabla. El agente no aprende mal: no puede ver.

(c) $\gamma < 1$ solo ayuda cuando la representación está rota. Con rangos amplios, $\gamma=0.995$ y $\gamma=1$ dan lo mismo (354.60 contra 354.83, indistinguibles). Con topes chicos, **$\gamma=0.995$ es 21.9 puntos mejor que $\gamma=1$.**

La explicación es incómoda y vale la pena decirla: con una representación que no distingue el endgame, la política correcta es imposible de expresar, y una política miope —cobrar temprano— es lo mejor disponible. El descuento hace al agente miope. O sea que **$\gamma < 1$ estaba tapando una falla de representación, no resolviendo un problema de horizonte**. Si uno solo mira el número, concluye " $\gamma=0.995$ funciona mejor" y se lleva la lección exactamente al revés.

3. De dónde sale el 81.5 %: descomposición

Componente	Aporte
Potencial Φ solo (representación, cero aprendizaje)	490.4
+ aprendizaje TD sobre afterstates	+36.7
Total	527.1

Esta descomposición es incómoda a propósito: **la mayor parte del puntaje viene del diseño de la representación, no del aprendizaje**. Sin el control " Φ solo" no habría forma de saberlo, y sería fácil atribuirle al algoritmo un mérito que es del modelado. Reportarlo es parte del trabajo.

Al mismo tiempo, el control inverso muestra que Φ solo tampoco alcanza: apagar Φ y aprender V directo hunde al agente a 105 puntos. Φ y el aprendizaje se necesitan mutuamente.

4. Arrepentimiento: dónde se pierden los 119 puntos que faltan

Con el oráculo se puede calcular, para cada decisión, cuánto valor destruyó exactamente:

$$\text{arrepentimiento}(o, a) = V^*(o) - [r(o, a) + E V^*(s')] \geq 0, = 0 \text{ si la jugada es óptima}$$

Perfil del agente entregado (300 partidas, banda desarrollo):

Bloque de turnos	Arrepentimiento acumulado
1 – 6	52.8
7 – 13	28.6
14 – 22	9.9
23 – 29	26.7
30	0.0
total	118.0

Errores más frecuentes:

Debería	Eligió	Veces
STORE_BEST_DIE	PASS	1022
BUY_SHIELD	STORE_BEST_DIE	895
BUY_SHIELD	SCORE	582

Y el efecto sobre la partida:

	dados (t=30)	bonus (t=30)	escudos (t23–29)	oro final
óptimo	7.61	6.17	0.73	483.6
agente	6.94	5.52	0.01	183.1

5. Por qué no cierra: dos causas, y ninguna es "faltan episodios"

5.1 Un óptimo local de política, no de valores

El agente casi nunca compra el escudo del turno 23, y por eso tampoco acumula. Las dos cosas se sostienen mutuamente: **el escudo solo rinde si después acumulás, y acumular solo conviene si tenés escudo**. Salir de ahí exige cambiar dos decisiones a la vez. ϵ -greedy explora desviaciones de un paso: prueba el escudo, ve que con su política actual no sirve, y lo descarta.

Se probaron tres esquemas de exploración pensados para este problema. Ninguno rompió el óptimo local:

Exploración	Mejor resultado
ϵ -greedy fijo ($\epsilon = 0.15$)	523.5
ϵ -greedy fijo ($\epsilon = 0.30 / 0.50$)	513.2 / 515.9
ϵ por episodio, log-uniforme (estilo Ape-X)	520.9
ϵ -z-greedy, exploración temporalmente extendida (Dabney et al., 2020)	503.8

Es un resultado negativo y se reporta como tal.

5.2 El límite es de resolución, no de razonamiento

En el estado típico donde el óptimo compra el escudo (t=23, 104 de oro, 7 dados, bonus 6):

Acción	V* (oráculo)	V (agente)	error
BUY_SHIELD	600.00	571.06	–28.94
STORE_BEST_DIE	594.74	576.63	–18.11
SCORE	594.24	573.21	–21.03
BUY_DICE	592.09	570.21	–21.88
UPGRADE	587.84	562.89	–24.95
PASS	586.44	568.47	–17.97

El óptimo separa la mejor acción de la peor por 13.6 puntos sobre un valor de 600: un 2.3 %. El error de estimación del agente es de 18 a 29 puntos — más grande que la diferencia entera que tiene que detectar.

O sea: el agente no razona mal. Su función de valor es correcta al 4 %, y la decisión pide 2 %.

Es un límite estadístico y se puede escribir como tal. Con muestras de desviación típica $\sigma \approx 130$ puntos, distinguir una diferencia de 13.6 con 95 % de confianza pide del orden de

$$n \approx (2 \cdot 1.96 \cdot 130 / 13.6)^2 \approx 1.400 \text{ muestras independientes por par de estados}$$

El agente tiene ~180.000 pesos y ~45 millones de actualizaciones: unas **250 por peso**. Falta un factor de aproximadamente **seis**. Eso da una predicción concreta y falsable: seis veces más muestras, o seis veces menos pesos, deberían alcanzar para esta decisión en particular. Las dos corridas que la prueban (5 millones de episodios, y grilla de oro reducida de 173 a 114 nodos) están documentadas en §6.

6. Corridas de verificación

La cuenta de §5.2 predice que **seis veces más muestras, o seis veces menos pesos**, deberían alcanzar. Las dos mitades de la predicción se probaron por separado:

Corrida	Qué prueba	Resultado
dbl_LONG — 5.000.000 de episodios	más muestras	falló. Meseta en 81 % desde los 500 mil episodios, y después baja. Triplicar los episodios no mueve la aguja.
dbl_COARSE — grilla de 114 nodos en vez de 173	menos pesos	parcial. 527.06 contra 523.92: +3.1 puntos, la dirección correcta pero un orden de magnitud menos de lo predicho.

Por qué falló la mitad de las muestras

Porque la cuenta supone muestras **representativas**, y no lo son. Los estados que deciden el resultado —turno 24, mucho oro, un escudo puesto— se visitan solo cuando la exploración compra el escudo, y al turno siguiente la política greedy vuelve a no acumular. Triplicar los episodios triplica unas muestras que nunca contienen la continuación correcta.

O sea que el cuello de botella no es el **tamaño** de la muestra sino su **distribución**. Es el problema de exploración profunda de §5.1, visto desde el otro lado: no se puede estimar el valor de una rama que la política de comportamiento nunca recorre hasta el final.

Es un resultado negativo, y es el más útil del trabajo: descarta "entrenar más" como camino y deja como única salida cambiar *qué se visita*, no *cuántas veces*.

7. Verificación contra sobreajuste

Todas las decisiones —hiperparámetros, elección de checkpoint, diseño de la representación— se tomaron mirando **una sola** banda de semillas. El agente final medido en las tres, $n = 20.000$ cada una:

Banda	Semilla base	Media	IC 95 %	% del óptimo
desarrollo — donde se decidió todo	1.000.000	527.06	[525.16, 528.97]	82.0 %
control — mirada una sola vez, al cerrar	2.000.000	526.27	[524.38, 528.15]	81.9 %
pública — la del enunciado	0	525.42	[523.53, 527.31]	81.8 %

Diferencia desarrollo – control: **+0.80**, con error estándar de la diferencia 1.37. Está dentro del ruido, así que no hay evidencia de sobreajuste a la banda de desarrollo.

Recordar que las semillas **no** son comparables entre agentes (01 REGLAS OCULTAS.md §8): el generador de datos y el de tormentas es el mismo, y `size=num_dice` depende de la política. No existen números aleatorios comunes en este ambiente, así que la única defensa contra el ruido es el tamaño de muestra.